

# Functional Programming

Radu Răzvan Slăvescu

Technical University of Cluj-Napoca  
Department of Computer Science

# Outline

Functors

Applicatives

# Functions generic over a range of parameterized types

## Abstracting out common patterns

- ▶ mapping: Functor
- ▶ function application: Applicative
- ▶ effectful programming: Monads

## Maybe: a new type for dealing with context

```
data Maybe a = Nothing | Just a
    deriving (Show)
```

## Observations

- ▶ `Just` can be regarded as a "wrapper" or as a context
- ▶ A value of type `Maybe a` either "wraps" a value of type `a` (in case of `Just a`) or is empty (in case of `Nothing`)
- ▶ This way, we can convey the idea of success or failure
- ▶ Applying functions on "wrapped" values could lead to different results, depending on the "wrapper" (i.e., context)
- ▶ `deriving (Show)` makes `Maybe` showable: a value of this type can be converted into a string and displayed
- ▶ Haskell can use its type system to build **stateful** programs (we'll see soon how this is done via Monads)

# class: introducing a new typeclass

## Definition (Typeclass)

A **set of types** that support some methods (i.e., behavior)

## Example (Introducing the class $\text{Eq}$ of equality types)

--this is how we specify a typeclass

```
class Eq a where
```

```
  -- declare its methods + their types
```

```
  (==), (/=) :: a -> a -> Bool
```

```
  (x /= y) = not (x == y)
```

## Remark

If type  $a$  wants to be an instance of class  $\text{Eq}$ , then it must provide an operator of the specified type for testing equality (one for inequality testing is provided by default).

## class: introducing a new typeclass

### Example (Make `Bool` into an equality type)

```
instance Eq Bool where
  False == False = True
  True   == True   = True
  False == True    = False
  True   == False  = False
```

### Remarks

- ▶ we have simply provided an implementation for `(==)`
- ▶ actually, `Bool` **uses** `deriving` to be made into an `Eq` type:

```
data Bool = False | True deriving (Eq)
```

# Functors in a nutshell

## Definition (What is a Functor?)

The class of types which support **mapping** on various structures (i.e., not solely on lists; it is more general).

```
class Functor f where
  fmap :: (a -> b) -> f a -> f b
```

## Remarks

If we want the parameterized type  $f$  to be an instance of class `Functor`, we must provide an implementation for the `fmap`. This takes a function  $a \rightarrow b$  and a structure of type  $f$  having elements of type  $a$ , applies the function to each element and builds a structure of type  $f\ b$  (with elements of type  $b$ ).

# Functors in a nutshell

## Making Maybe into a Functor: implement fmap

```
instance Functor Maybe where
  -- implement the required method(s)
  fmap g (Just val) = Just (g val)
  fmap g Nothing    = Nothing
```

## Example

```
> fmap (+1) (Just 42)
Just 43
```

## What can a Functor do?

A Functor knows how to apply a function to a wrapped value.



# Functors in a nutshell

## Making list into a Functor

```
instance Functor [] where    --[] a same as [a]
    --fmap :: (a -> b) -> [a] -> [b]
    fmap = map
```

## Example (Using fmap on lists)

```
> fmap (+11) [1,2,3]
[12,13,14]
```

## Example (fmap in infix style: <\$>)

```
> (+11) <$> [1,2,3]
[12,13,14]
```

# Functors in a nutshell

## Making `function` into a `Functor`

```
instance Functor ((->) a) where
  --fmap :: (b -> c) -> (a -> b) -> (a -> c)
  fmap g h = g . h
```

## Example

```
> (fmap (*1.2) (+100)) 1000
1320.0
```

# Functors in a nutshell

## Don't Repeat Yourself: generic functions

```
inc :: Functor f => f Int -> f Int  
inc = fmap (+1)
```

## Example

```
> inc [10,20,30]  
[11,21,31]  
> inc (Just 100)  
Just 101  
> inc (*3) 12  
37
```

## Remark

We can define generic functions, which work with any functor

# Functors in a nutshell

**Functor laws:** `fmap` must preserve `id` and composition

$$\text{fmap } \text{id} = \text{id}$$
$$\text{fmap } (g \ . \ h) = \text{fmap } g \ . \ \text{fmap } h$$

## Remark

For any parameterized type in Haskell, there exists at most one function `fmap` which obeys these laws.

# Applicatives in a nutshell

## Example (Map a function of k arguments)

```
fmap2 :: (a->b->c) -> f a -> f b -> f c
```

```
fmap3 :: (a->b->c->d) -> f a -> f b -> f c -> f d
```

...

## Remark

One Functor per fmap? How may? How about currying?

We need a way to map functions with any number of arguments over a structure

# Applicatives in a nutshell

## What is an Applicative (aka Applicative Functor)?

Something which allows mapping functions with *any* number of arguments. To achieve this, it relies on currying + two functions:

```
pure :: a -> f a
(<*>) :: f (a->b) -> f a -> f b  --"applied over"
```

## Remarks

1. `pure` converts a value of type `a` into a structure of type `f a`
2. `<*>` generalizes function application: the argument function, the argument value and the result value are all contained in `f` structures.

# Applicatives in a nutshell

## Writing `fmap2`, `3`, ... in applicative style

```
fmap1 :: (a->b) -> f a -> f b
```

```
fmap1 g x = pure g <*> x
```

```
fmap2 :: (a->b->c) -> f a -> f b -> f c
```

```
fmap2 g x y = pure g <*> x <*> y
```

```
fmap3 :: (a->b->c->d) -> f a -> f b -> f c -> f d
```

```
fmap3 g x y z = pure g <*> x <*> y <*> z
```

# Applicatives in a nutshell

## Defining Applicatives: `pure` + `<*>` + currying

```
class Functor f => Applicative f where
  pure  :: a -> f a
  (<*>) :: f (a->b) -> f a -> f b
```

## Example (Let's make `Maybe` an Applicative)

```
instance Applicative Maybe where
  pure          = Just
  Nothing <*> _  = Nothing
  (Just g) <*> mx = fmap g mx
```



# Applicatives in a nutshell

## Example (Using the `Maybe` Applicative)

```
pure           = Just
Nothing  <*> _   = Nothing
(Just g) <*> mx  = fmap g mx
```

```
> pure (+1) <*> Just 42
Just 43
```

```
> pure (+) <*> Just 42 <*> Just 43
Just 85
```

```
> pure (+) <*> Nothing <*> Just 43
Nothing
```

# Applicatives in a nutshell

## Example

```
> Just (+10) <*> (Just 22)
```

```
Just 32
```

```
> (+) <$> Just 5 <*> Just 20
```

```
Just 25
```

```
> ((+) <$> Just 5) <*> Just 20
```

```
Just 25
```

```
> (+) <$> (Just 5 <*> Just 20)
```

```
error:
```

```
Non type-variable argument in the constraint: Num (a -> a1)
```

## What can an Applicative do?

It knows how to apply a wrapped function to a wrapped value.  
(a simple `Functor` can't do that).

# Applicatives in a nutshell

## Let's make `List` an Applicative

```
instance Applicative [] where
  pure x      = [x]
  gs <*> xs = [ g x | g <- gs, x <- xs ]
```

## Remarks for the `List` Applicative

`pure` takes a value and transforms it into a singleton list.  
`<*>` applies each function in `gs` to each argument in `xs`

# Applicatives in a nutshell

## Example (Using the `List` Applicative)

```
pure x      = [x]  
gs <*> xs = [ g x | g <- gs, x <- xs ]
```

```
> pure (+2) <*> [10,20,30]  
[12,22,32]
```

```
> pure (+) <*> [2] <*> [42]  
[44]
```

```
> pure (+) <*> [2] <*> [42,43]  
[44,45]
```

```
> pure (*) <*> [2,3] <*> [5,7]  
[10,14,15,21]
```

# Applicatives in a nutshell

## Remarks

1. Applicatives abstract on applying pure functions to effectful arguments (e.g., which can fail, perform IO actions etc)
2. Applicative `Maybe` supports *exceptional* programming: pure functions are applied to arguments that may fail. Failure propagation is handled automatically
3. Applicative lists support *non-deterministic* programming: pure functions can be applied to multi-valued arguments. Combining values and failure propagation are performed automatically

# Applicatives in a nutshell

## Applicatives laws

`pure id <*> x` = `x`

`pure (g x)` = `pure g <*> pure x`

`x <*> pure y` = `pure (\g -> g y) <*> x`

`x <*> (y <*> z)` = `(pure (.) <*> x <*> y) <*> z`

## Remark

These laws ensure that every well-typed expression built using `pure` and the `<*>` operator can be written in applicative style:

`pure g <*> x1 <*> ... <*> xn`

# Applicatives in a nutshell

## Definition (`liftA2`)

```
liftA2 :: Applicative f => (a -> b -> c) ->
                                     f a -> f b -> f c
liftA2 f x y = f <$> x <*> y
```

## Remark

`liftA2` simplifies writing functions in applicative style: we have to specify only the (binary) function and the argument lists

# Applicatives in a nutshell

## Definition (`liftA2`)

```
liftA2 :: Applicative f => (a -> b -> c) ->
                                         f a -> f b -> f c
liftA2 f x y = f <$> x <*> y
```

## Example (Solution with no `liftA2`)

```
> pure (\ f t -> f ++ "2" ++ t) <*> ["B", "C"]
                                   <*> ["B", "C"]
["B2B", "B2C", "C2B", "C2C"]
```

## Example (Solution which uses `liftA2`)

```
> liftA2 (\ f t -> f ++ "2" ++ t) ["B", "C"]
                                   ["B", "C"]
["B2B", "B2C", "C2B", "C2C"]
```



# Applicatives in a nutshell

## Definition (`sequenceA`)

```
sequenceA :: Applicative f => [f a] -> f [a]
sequenceA []      = pure []
sequenceA (x:xs) = pure (:) <*> x
                  <*> sequenceA xs
```

## Remark

`sequenceA` transforms a list of applicative actions into one applicative action which returns a list of result values. Functions like this can be used with *any* Applicative

## Example (`sequenceA`)

```
> sequenceA [Just 11, Just 12, Just 13]
Just [11, 12, 13]
```

# Summary

- ▶ Functors can map functions on arbitrary structures
- ▶ Applicatives help applying function with arbitrary numbers of arguments
- ▶ They pave the way to effectful programming in a controlled manner